

Agentic RAG는 가치가 있을까요? RAG 접근 방식 비교 실험

피에트로 페라치^{1,2}, 밀리카 Cvjeticanin³, 알레시오 파라치니⁴, 다비데 지아누치⁵,
 1Fondazione Bruno Kessler, 트렌토, 이탈리아, 2Università di Padova, 파도바, 이탈리아,
 3. 카길 제네바, 스위스, ⁴ 알케미, 밀라노, 이탈리아, 5 소속 5

연락처: pferrazzi@fbk.eu

추상적인

검색 증강 생성(RAG) 시스템은 일반적으로 다음과 같은 조합으로 정의됩니다.
 발전기와 회수 구성 요소의
 지식 기반에서 텍스트 컨텍스트를 추출합니다.
 사용자 문의에 답변하기 위해서입니다. 하지만 이러한 기본적인 구현 방식에는 몇 가지 한계가 있습니다.
 잡음이 많거나 최적화되지 않은 검색, 범위를 벗어난 쿼리에 대한 검색의 오용, 취약한 검색 등을 포함합니다.
 질의-문서 일치 및 가변성 또는
 발전기와 관련된 비용. 이러한 단점들이 발전기 개발을 촉진했습니다.
 전용 모델이 포함된 "향상된" RAG
 특정 약점을 해결하기 위해 도입되었습니다.
 워크플로우에서. 최근에는 대규모 언어 모델(LLM)의 자기 성찰 기능이 향상되면서 새로운 방식이 가능해졌습니다.
 다.
 우리가 "에이전트" RAG라고 부르는 패러다임입니다.
 이 접근 방식에서 LLM은 전체 프로세스를 총괄합니다.
 즉, 어떤 작업을 수행할지, 언제 수행할지, 그리고 반복할지 여부를 결정함으로써 고정된 수동 설계 모듈에 대한 의존도를 줄입니다. 이러한 빠른 속도에도 불구하고

두 패러다임을 모두 채택하더라도 여전히 불분명한 점이 남아 있습니다.
 어떤 조건에서 어떤 접근 방식이 더 바람직한가. 본 연구에서는 광범위한 분석을 수행한다.
 향상된 및 경험적 평가에 대한 경험적 근거에 기반한 평가
 다양한 시나리오와 차원에 걸친 에이전트 기반 RAG.
 본 연구 결과는 실질적인 통찰력을 제공합니다.
 두 패러다임 사이의 장단점을 살펴보겠습니다.
 실제 적용 분야에 가장 효과적인 RAG 설계를 선택하는데 필요한 지침을 제공합니다.
 비용과 성능을 모두 고려합니다.

1. 서론

대규모 언어 모델(LLM)은 자연어 이해 및 생성 작업에서 놀라운 결과를 보여주었습니다. 그러나,

기존 알고리즘들은 고정된 학습 데이터에 의해 기능이 제한되어 왔기 때문에, 민간 기업의 지식을 LLM 기반 시스템에 통합해야 할 필요성이 대두되었습니다. 검색 증강 생성(RAG)은 이러한 필요성을 충족시켜 줍니다.

실용적인 해결책으로 떠오른 것은 바로 매개변수적 지식에만 의존하는 대신, 외부 지식 기반에서 관련 문서를 검색하고 이를 바탕으로 모델을 생성하는 방식입니다.

증거 (Lewis et al., 2020). 이 방법은 연구 커뮤니티에서 상당한 주목을 받았습니다 (Wang et al., 2024; Fan et al., 2024; Wang).

et al., 2024) 및 산업 분야에서 클라우드 제공업체들이 자체 RAG 솔루션을 제공하고 있습니다 (IBM, 2025; Amazon Web Services, 2025; Microsoft Azure, 2025).

이 최초의 초기 정의 이후 RAG 워크플로우

이러한 시스템은 우리가 Enhanced RAG 라고 정의하는 것으로 확장되었습니다 (그림 1, 왼쪽). 이러한 시스템은 다음을 추가합니다.

검색 및 생성 블록 구성 요소는 다음과 같습니다.

쿼리 재작성과 같은 추가적인 정제 작업을 수행합니다.

문서 재순위화 및 쿼리 라우팅. 최근에,

LLM(법학 석사)의 자기 성찰 능력이 향상됨에 따라 에이전트형 RAG로의 전환을 가능하게 했습니다 (그림 1).

오른쪽)에서 LLM은 오케스트레이터 역할을 하며, 수행할 작업을 결정하고, 다양한 목적에 따라 다양한 도구를 활용할 수 있습니다.

시스템은 더 이상 고정된 파이프라인이 아니라, 순서가 미리 정해지지 않은 반복 루프에서,

모델이 모든 결정을 담당합니다. 비록

이론적 차이점을 규명하기 위한 초기 작업

향상된 RAG 시스템과 에이전트형 RAG 시스템 사이에는 다음과 같은 차이점이 있습니다.

(Neha와 Bhati, 2025)가 제안했지만, 여전히 다음과 같은 문제가 남아 있습니다.

두 시스템 간의 성능 차이가 무엇인지 불분명했습니다. 따라서 우리는 다음과 같은 실험을 수행했습니다.

서로 다른 영역에서 두 방법을 실험적으로 비교하는 것.

우리의 첫 번째 기여는 평가입니다.

문헌에서 발췌한 네 가지 차원에서 두 시스템의 성능을 비교했습니다 (표 1).

기존 RAG 시스템의 약점을 파악하고, 이전 연구에서 이러한 약점이 어떻게 해결되었는지 분석했습니다.

본 연구에서는 향상된 구현과 에이전트 구현을 비교하기 위한 실험적 환경을 제안했습니다.

선택된 치수는 다음과 같습니다.

1. 사용자 의도 처리, 시스템 성능 측정

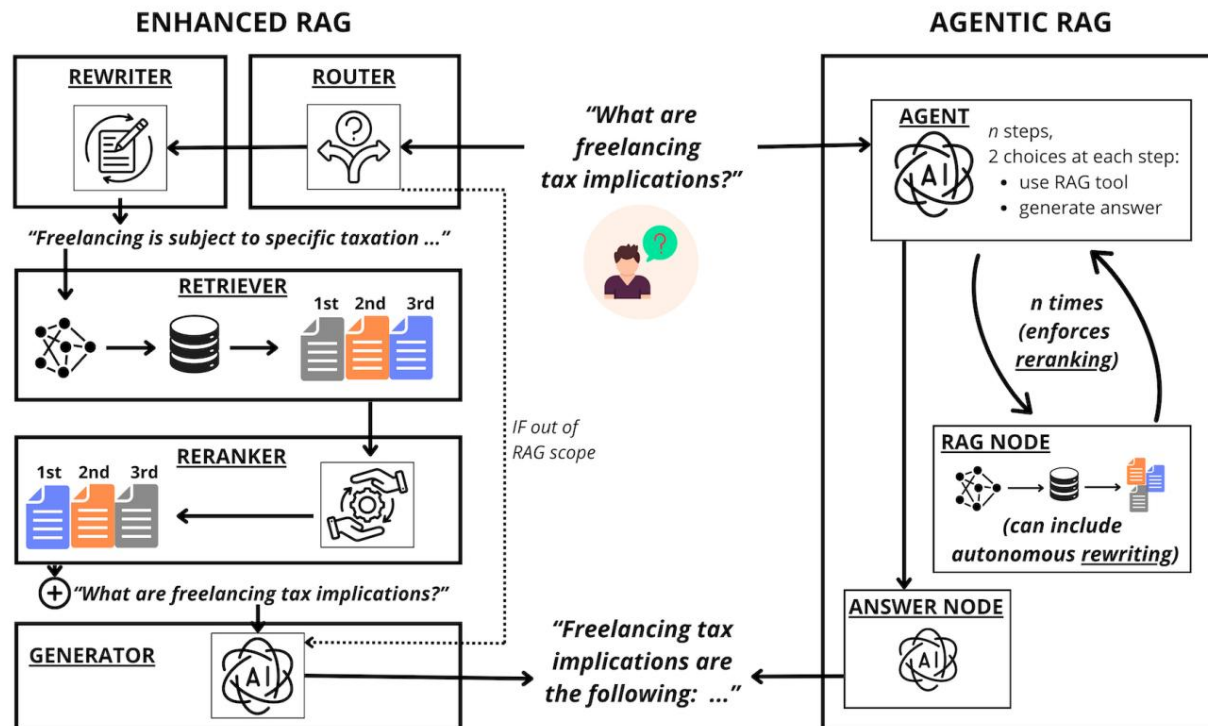


그림 1: 왼쪽 — 향상된 RAG. 이 시스템은 일련의 모듈로 구성되며, 각 모듈은 RAG 파이프라인의 특정 단계를 개선하는 역할을 합니다. 라우터는 쿼리가 검색을 트리거해야 하는지 여부를 결정하고, 리라이터는 쿼리를 재구성하며, 리트리버는 지식 베이스에서 후보 청크를 선택하고, 리랭커는 검색된 컨텍스트를 생성기에 전달하기 전에 순서를 정합니다. 워크플로는 고정되어 있습니다. 정보는 단순한 RAG 시스템(리트리버와 생성기 블록의 단순한 구성으로 정의됨)의 알려진 약점을 완화하기 위해 미리 정의된 블록을 통해 흐릅니다. 오른쪽 — 에이전트 기반 RAG. LLM은 전체 프로세스를 조율하는 에이전트 역할을 합니다. 각 단계에서 RAG 도구를 호출할지 또는 답변 생성으로 직접 진행할지 선택할 수 있습니다. 검색 및 컨텍스트 정제는 반복될 수 있으며, 에이전트는 작업의 변화하는 상태에 따라 작업을 자율적으로 선택하고 순서를 정합니다. 이를 통해 미리 정의된 중간 모듈이 없는 반복적인 파이프라인이 구현됩니다.

용어는 외부 지식이 필요한 쿼리와 필요하지 않은 쿼리를 구분합니다.

2. 질의-문서 정렬: 시스템이 질의를 지식 기반 문서 형식에 맞추는 능력을 측정합니다.
3. 검색된 문서 조정: 검색 후 문서 선택을 더욱 세밀하게 조정할 수 있는 시스템의 능력을 측정합니다.
4. LLM 품질의 영향, 즉 기본 모델의 기능 변화에 대한 시스템의 견고성을 측정합니다.

두 번째 기여는 여러 시나리오에서 두 시스템에 필요한 비용과 계산 시간을 자세히 분석하는 것입니다. 마지막으로, 연구자와 실무자들이 필요에 가장 적합한 시스템을 선택하는 데 도움이 되도록 연구 결과를 간략하게 요약했습니다.

2. 검색 증강 생성

검색 증강 생성(RAG)이라는 용어는 문헌 전반에 걸쳐 일관성 없이 사용되는 경우가 많습니다. 본 연구에서 사용된 정의들을 소개합니다.

RAG 정의: RAG 시스템은 i) 텍스트 덩어리로 구성된 지식 기반(KB), ii) 쿼리가 주어졌을 때 지식 기반에서 관련 덩어리를 추출하는 검색 방법, iii) 쿼리와 검색된 덩어리를 모두 고려하여 답변을 생성하는 생성기의 통합체로 정의합니다.

단순 RAG (그림 1)는 RAG 패러다임의 가장 단순한 구현체로, 생성기는 대규모 언어 모델이며 워크플로는 네 가지 순차적인 단계로 구성됩니다. i) 쿼리가 수신되면, ii) 검색기가 지식 베이스(KB)에서 고정된 개수의 청크를 결정론적으로 선택하고 (검색), iii) 검색된 청크를 쿼리 앞에 추가하고, iv) 생성기에 전달하여 응답을 생성합니다(생성).

순진한 RAG의 단점	우리가 평가하는 것	구현	
		향상된	에이전트
검색은 다음과 같은 경우에도 수행됩니다. 필요하지 않은 쿼리	RAG 사용의 정확도 범위 내 및 범위 외 쿼리	의미론적 라우팅 체계	에이전트가 결정합니다 할지 말지 검색 여부
지식 베이스(KB)의 질의 및 문서 형식이나 의미론이 다릅니다. 검색 성능 저하를 초래함	쿼리 재작성의 영향 기법	Hyde 기반 쿼리 다시 쓰기	에이전트가 다시 씁니다 원하는 대로 쿼리하세요
잡음이 많거나 최적화되지 않은 검색	검색된 문서의 영향 리스트 정제 기법	인코더 기반 재순위 지정자	에이전트가 다시 할 수 있습니다 검색 다중 타임스
기초 LLM이 너무 약합니다. 시간이 너무 많이 걸린다 / 너무 값비싼	더 많이/더 적게 선택하는 것의 영향 강력한 모델	다양한 LLM을 테스트해 보세요.	

표 1: 우리가 선택한 평가 차원의 요약. Naïve RAG의 각 단점에 대해 다음과 같이 정의합니다.
평가 차원("무엇을 평가하는가")과 향상된 에이전트형 RAG가 어떻게 작동하는지 테스트하기 위한 구현 방안
그러한 한계를 극복하십시오.

Huang 및 Huang (2024) 에서 제시된 분류 체계를 따라 , 향상
된 RAG (그림 1, 왼쪽)를 모든 단순 RAG로 정의합니다.

추가 단계를 통해 강화된 파이프라인
성능을 향상시키기 위해 이러한 개선 사항이 적용되었습니다.
워크플로의 여러 단계에서 발생할 수 있습니다.
다양한 방식으로 구현되며, 이에 대해서는 아래에서 논의하겠습니다.
자세한 내용은 관련 섹션을 참조하십시오.

에이전트형 RAG 우리는 에이전트형 RAG를 정의합니다(그림 1,
(맞습니다) LLM이 워크플로우를 제어하고 동적으로 조정할
수 있는 시스템입니다.
행동을 수행하기로 결정합니다. 이러한 에이전트 제어를 통해 시스
템은 검색-생성 과정을 조정할 수 있습니다.
특정 컨텍스트와 당면한 작업에 맞춰 루프를 실행합니다. 향상된
RAG와 달리 기본 지식 기반 검색기-생성기 설정과 관련하여 추가
구성 요소가 도입되지 않습니다.

3. 관련 연구

RAG 개념은 최초로 도입된 바 있다.
(Lewis et al., 2020)은 집중적인 연구를 거쳐 빠르게 발전하여
매우 정교한 수준에 이르렀습니다.
시스템. 포괄적인 개요가 제시됩니다.
Gao et al. (2023b) 이 제안한 설문조사에 따르면 ; Fan
et al. (2024); Wang et al. (2024). 대규모 언어로서
LLM 모델들은 단단계 추론 및 성찰 능력을 갖추게 되었으며, 그
일관성은 다음과 같습니다.
Agentic RAG 솔루션 으로의 패러다임 전환을 가능하게 했습니
다 (Shinn et al., 2023; Madaan et al., 2023).
인공지능 에이전트를 특징짓는 속성에 대한 정의는 진화해 왔지만
(Masterman et al., 2024),
이러한 새로운 연구 방향은 아직 제대로 정립되지 않았습니다.
Singh 등이 초기 시도를 한 것을 시작으로, 통합된 분류 체계 내에
서 포괄적으로 분류되었다 .
(2025); de Aquino 및 Aquino et al. (2025).

에이전트 설계 및 구현 여러 가지
최근 오픈소스 프레임워크가 등장했습니다.
에이전트 기반 RAG 시스템 개발을 지원합니다.
급속한 성장과 실험을 반영합니다
이 영역에서 SmolAgents (Roucher et al., 2025)
LangGraph (LangChain Inc., 2025), LlamaIndex
(Liu, 2022), CrewA (기여자, 2025a), Auto- Gen (Microsoft,
2025), PydanticAI (기여자,
2025b), 그리고 Atomic Agents (BrainBlend-AI, 2025)
각각 다른 장점을 제공하며 다양한 설계 선택을 반영합니다.
우리는 경량 프레임워크인 PocketFlow를 사용합니다.
또한, 단순한 그래프 기반 추상화를 제공하고 복잡성을 피하는 최
소한의 프레임워크입니다.
더 큰 규모의 라이브러리를 통해 깨끗하고 통제된 환경을 보장합니다.
평가. 부록의 표 8 에 요약되어 있습니다.
우리가 선택한 프레임워크입니다.

그럼에도 불구하고 경험적 비교의 필요성
개념적, 건축적 측면에서 급속한 발전
검색 증강 생성(RAG) 분야에서 에이전트 기반 RAG와 향상된 RAG
간의 포괄적인 실험적 비교 연구는 아직 문헌에서 부족한 실정입니
다.
RAG 시스템. 이 방향에 대한 예비 연구는 Neha와 Bhati (2025)
가 제시했습니다.
유용한 정의 및 평가 방법을 제안합니다.
차원을 고려하지만 완전한 실증 연구를 수행하지는 않습니다 .

4 데이터

실험을 수행하기 위해서는 데이터 세트가 필요합니다.
이는 일반적인 RAG 애플리케이션을 대표하는 것으로, 질의와 지식
기반이 쌍을 이루는 형태입니다. 제안된 분류 체계를 따릅니다.
Arslan et al. (2024) 에 따르면 RAG는 다음과 같이 분류됩니다.
응용 분야별 사용 사례를 중심으로 살펴보고겠습니다.
가장 두드러진 자연어 기반 시나리오-

iOS를 기준으로, 우리는 크게 두 가지 범주를 고려합니다. i) 질문 답변(QA): RAG를 활용하여 사실적 지식에 기반한 답변을 제공하는 경우, ii) 정보 검색 및 추출(IR/E): RAG를 자연어 쿼리를 통해 데이터에서 지식을 추출하는 도구로 사용하는 경우입니다 .

QA를 위해 Thakur et al. (2021) 이 제안한 버전의 FIQA (Maia et al., 2018) 와 NQ (Kwiatkowski et al., 2019) 를 선택했습니다 . IR/E를 위해 FEVER (Thorne et al., 2018) 와 CQADupStack-English (Hoogeveen et al., 2015) 를 사용했습니다 .

각 데이터 세트는 서로 다른 실제 시나리오를 나타내며 , 서로 다른 테스트 작업을 수행할 수 있도록 선택되었습니다(표 2).

- FIQA(금융 질의응답)는 질의가 전문가 지식을 바탕으로 답변을 제공해야 하는 도메인별 질문인 RAG 사용 사례를 나타냅니다 . 이 작업은 사용자의 질의에 대한 답변을 제공하는 것으로 구성됩니다.
- NQ(자연스러운 질문)는 사용자가 어떤 유형의 질문이든 할 수 있는 광범위한 QA 사용 사례를 포괄합니다. 과제는 사용자의 질문에 가장 적절한 답변을 제공하는 것입니다.
- FEVER는 법률 및 생물의학 분야에서 흔히 사용되는 주장 검증 작업을 테스트합니다 . 사용자는 특정 주장에 대한 찬반 증거를 찾고, 이를 뒷받침하거나 반박하는 문서를 찾아 최종 평가 결과와 참고 자료 요약물을 제공합니다.
- CQADupStack-English는 RAG를 사용하여 이전에 해결된 티켓과 토론을 찾는 설정을 나타냅니다 . 작업은 다음과 같이 구성됩니다.

사용자가 제기한 질문과 동일한 내용을 다루는 기존 블로그 게시물을 찾아 사용자가 이해하기 쉬운 요약을 제공합니다.

5. 평가

RAG 시스템 평가는 종종 개별 하위 구성 요소에 대한 평가로 분해되어 이루어지며 (Es et al., 2024; Chen et al., 2020), 각 하위 구성 요소는 전반적인 효과에 영향을 미치는 차원에 해당합니다 . 본 연구에서도 동일한 접근 방식을 따라 평가를 설계했습니다. 먼저 Huang과 Huang (2024) 의 연구를 기반으로 단순 RAG 파이프라인의 한계 목록을 도출한 다음 , 향상된 RAG와 에이전트형 RAG의 두 가지 구현을 비교하는 실험 환경을 구축했습니다.

작업	도메인 데이터셋 #쿼리 #문서				평균 디/큐
QA 일반	NQ 3,452 2,681,468 1.2 57,638 2.6	금융 FIQA	648		
IE/R	문법 포럼	CQAD-영어	1,570	40,221	1.4
	위키피디아 열병		6,666 5,416,568 1.2		

표 2: 실험 설정에 사용된 네 가지 데이터셋 개요. 질의응답 (QA)과 정보 추출 및 검색 (IE/R) 각각에 대해 두 개의 데이터셋이 선택되었다. 각 질의에는 관련 문서와 관련 없는 문서의 레이블이 지정된 목록이 있다 . 평균 D/Q는 질의당 관련 문서의 평균 개수를 나타낸다.

시스템. 표 1은 단순 RAG의 주요 단점을 요약하고 향상된 RAG와 에이전트형 RAG가 이러한 단점을 어떻게 해결하는지 보여줍니다.

식별된 네 가지 차원 각각에 대해 i) 해당 차원을 정의하고, ii) 두 시스템 모두 해당 차원을 다룰 수 있도록 구현 방식을 상세히 설명하고, iii) 평가 환경을 제시하고, iv) 평가 지표를 정의합니다.

5.1 사용자 의도 처리

정의: 본 연구에서 사용자 의도 처리란 특정 쿼리에 검색 기능 사용이 필요한지 여부를 판단하는 것을 의미합니다 .

RAG에 대한 기존 연구 (Huang and Huang, 2024; Gao et al., 2023b; Fan et al., 2024) 에서는 의도 감지를 명시적으로 다루거나 언급 하지 않았지만 , Wang et al. (2024) 은 이 작업을 위한 전용 분류기를 제안하여 의도 감지의 중요성을 강조했습니다. 본 논문에서는 의도 감지가 실제 RAG 시스템에서 매우 중요하다고 주장합니다. 의도 감지는 불필요하거나 부적절한 검색 호출을 방지하고 , RAG가 질의에 의미 있게 답변하는 데 기여할 때만 호출되도록 보장하기 때문입니다 .

향상된 구현 우리는 의미론적 라우터 프레임워크'를 사용하여 향상된 RAG 라우팅 시스템을 구현합니다 .

라우터는 유효한 쿼리와 유효하지 않은 쿼리로 각각 레이블이 지정된 두 세트의 예제 쿼리로 정의됩니다. 추론 시 사용자 쿼리는 이 두 그룹과 비교되어 유효 또는 무효로 분류됩니다. 시스템은 RAG(Relational Assessment Group)를 사용하여 유효한 쿼리에 응답하고 유효하지 않은 쿼리에는 응답하지 않습니다. 본 실험에서는 OpenAI의 text-embedding-3-small 임베더를 사용했습니다 . 라우팅 시스템의 구조에 대한 자세한 내용은 부록 A.2에 제시되어 있습니다 .

에이전트 기반 구현 에이전트 기반 RAG 시스템 은 구별하는 능력을 내장합니다.

¹ <https://github.com/aurelio-labs/semantic-router.git>

	QA	분노	
환경	파카	열 CQA-EN	레크 F1
	레크 F1 레크	F1	레크 F1
순진한	100 66.7 100 66.7	100 66.7	
향상된	95.1 95.7 84.4 87.9 94.7 96.6		
에이전트	97.7 98.8 49.3 64.6 100 99.8		

표 3: 사용자 의도 처리 재현율 및 F1 점수
데이터 세트당 유효 및 무효 쿼리 500개 . 기준선
이는 각 사용자 쿼리에 대해 검색이 수행되는 단순 RAG로 표현됩니다 . 향상된 설정은 다음과 같습니다.
의미론적 라우터 접근 방식을 기반으로 하는 동안 에이전트는 RAG 도구를 사용할지 여부를 자율적으로 결정했습니다.

	QA	분노		
FIQA NQ FEVER CQAD AVG 설정				
순진한	45.3 45.8 46.3	66.2		
향상된	43.5	43.9	81.1	42.8 <u>52.8</u>
에이전트	43.2 51.7	83.1	44.3	55.6

표 4: 쿼리 재작성 성능 (다음 항목 포함)
NDCG@10. 기준선은 Naïve RAG로 표현됩니다.
사용자 쿼리가 아무런 제약 없이 직접 포함되어 있습니다.
더 나아가 변화.

의도적으로 검색이 필요한 쿼리.
문의가 접수되면 에이전트는 자유롭게 결정할 수 있습니다.
RAG 노드를 활용할지 아니면 직접 답변할지 여부.
저희는 gpt-4o를 기본 LLM으로 사용합니다.

실험 환경에서 성능을 테스트했습니다.
동일한 수의 요소로 구성된 데이터 세트에서
유효한 쿼리와 유효하지 않은 쿼리 (네 가지 유형 각각 500개씩)
데이터 세트). 우리는 유효한 쿼리를 선택했습니다.
각 데이터셋의 분할 학습을 진행하는 동안 우리는 다음과 같은 결과를 생성했습니다.
5-shot을 통해 gpt-4o를 유발하는 유효하지 않은 것들입니다. 우리는 만듭니다.
유효한 데이터셋과 유효하지 않은 데이터셋을 공개적으로 이용할 수 있습니다.
queries2 . 우리는 이 분석에서 NQ 데이터셋을 제외했습니다.
평가 단계는 설계상 모든 유형의 문제를 처리합니다.
쿼리를 통해 유효하지 않은 정의를 방지합니다.

평가 지표 시스템이 쿼리를 올바르게 처리하는지 평가하기 위해
F1 점수를 활용했습니다.
다시 말해, 양쪽 모두의 성능을 고려해야 한다는 것입니다.
유효한 클래스와 유효하지 않은 클래스.

5.2 쿼리 재작성

정의 많은 관심이 집중되어 왔습니다.
쿼리 재작성 기법은 (Ma) 에 의해 처음 소개되었습니다.
(et al., 2023). 핵심은 사용자가 쿼리할 때입니다.
지식 기반과 비교하여 유사성을 검사합니다.
검색 시, 비교는 종종 다음과 같은 것들 사이에서 수행됩니다.
상당히 다른 텍스트입니다. 질문은 보통 짧습니다.

²<https://huggingface.co/datasets/DLBDAlkemy/>
사용자 의도 처리

그리고 밀도 높은 질문인 반면, 지식 베이스의 덩어리는
길고 복잡할 수 있습니다. 쿼리 재작성 기법
쿼리를 다시 작성하여 이 차이를 줄이는 것을 목표로 합니다.
목표 텍스트와 구조가 더 유사한 텍스트
체크. Hyde (Gao et al., 2023a)는 다음과 같이 구성됩니다 .
해당 질문을 짧은 단락으로 대체합니다.
이에 대한 답변으로, 가장 우수한 성능을 보이는 기술 중 하나로 부상했습니
다 (Wang et al., 2024).

향상된 구현 우리는 향상된 RAG 시스템이 Hyde 쿼리 재작성
을 수행하도록 강제했습니다 . 각 사용자 쿼리는 자동으로 재작
성됩니다.
검색 단계를 수행하기 전에 gpt-4o 에 다음 내용을 입력하세
요 : 작성해 주세요
질문에 답하기 위한 지문입니다. 질문:
{user_query}\n 구절:".

에이전트 구현 우리는 프롬프트를 설계합니다
담당자에게 재작성이 도움이 될 수 있음을 알리기 위해서입니다.
에이전트가 RAG 도구를 사용하기로 선택하면,
다음과 같은 단계를 수행하도록 결정할 수 있습니다. "사용자
쿼리를 {type_of_doc} 형식으로 변환".
문서 유형은 데이터 세트에 따라 다릅니다("더 긴").
CQADupStack의 블로그 게시물 "답변을 위한 구절"
FiQA와 NQ의 경우 "it", "더 긴 사실 진술"
(발열의 경우)

실험 설정: 모든 쿼리를 실행합니다.
4개의 데이터셋 테스트 세트에서 2개의 데이터셋과 비교
시스템. 공정한 비교를 보장하기 위해, 그러한 경우에
Agentic 설정이 쿼리를 수행하지 않은 경우
재작성 과정에서 검색 지표를 계산합니다.
원래 검색어.

평가 지표: 네 가지 쿼리 모두
데이터 세트에는 정답에 대한 주석이 함께 제공됩니다.
연결되어야 할 문서들입니다. 검색된 청크의 품질을 평가할 때,
우리는 다음을 수행할 수 있습니다.
그다음 NDCG@10을 사용하십시오. NDCG는 정규화된 할인 누적 이득

(Järvelin 및
Kekäläinen, 2002) 이며 가장 흔한 것 중 하나입니다.
순위의 효과성을 평가하는 데 사용되는 지표
모델. 차단 주파수 K에서의 NDCG는 다음과 같이 정의됩니다.

$$\text{NDCG@K} \equiv \frac{\text{DCG@K}}{\text{maxDCG@K}}$$

(1)

여기서 maxDCG@K 는 최대 DCG@K입니다.
주어진 관련성 레이블로부터 얻을 수 있습니다.
그리고 DCG@K는 다음과 같이 정의됩니다.

$$\text{DCG@K} \equiv \sum_{i=1}^{\text{케이}} \frac{2^{\text{리}} - 1}{\log(1 + i)}$$

(2)

	QA	분노	
	FIQA CQA-EN 평균		
순진한	45.0	46.0	45.5
재작성 없이 항상된 재작성 에이전	49.0	47.0	48.0
트를 사용하여 항상됨	51.0	48.0	49.5
	43.4	44.4	43.9

표 5: 문서 목록 정제 성능

NDCG@10의 조건. 에이전트 설정은 다음을 수행합니다.

이 작업은 필요한 만큼 검색을 반복하여 수행합니다.

반면, 항상된 설정은 재 순위화 모델을 통해 이를 달성합니다. 에이전트 기반 검색은 간접적으로 쿼리 재작성을 수행하므로, 항상된 설정에 대해 재작성을 포함한 결과와 포함하지 않은 결과를 모두 보고하여 두 경우를 모두 고려합니다.

공정한 비교입니다.

여기서 li 는 관련성 레이블(실제 정답)입니다.

순위에서 i 번째 위치에 있는 문서의 레이블입니다.

방정식 5.2 는 항상 양수이므로, 방정식 5.2 는 0과 1 사이의 숫자입니다.

NDCG@K가 1과 같다는 것은 완벽한 상태임을 의미합니다. 순위 목록.

5.3 문서 목록 세분화

정의: 검색 단계에서 숫자를 선택합니다.

반드시 가장 적절한 것은 아닌 조각들로 이루어진 데이터 덩어리들. 이전 연구에서는 초기 데이터 덩어리들이 어떻게 작용하는지 보여주었습니다.

검색 결과에는 부분적으로 관련성이 없거나 노이즈가 포함된 데이터가 포함될 수 있습니다.

결과 및 개선 방안을 제안합니다.

재순위화 전략을 통한 선택 과정 (Sachan

등, 2022; Sun 외, 2023; 진 외., 2024).

재순위화는 검색된 데이터 덩어리를 정렬하는 것을 의미합니다.

사용자 쿼리와 관련 하여 관련성이 높은 항목들의 부분집합을 선택합니다.

항상된 구현 우리는 다음과 같은 실험을 진행합니다.

ELECTRA 기반을 사용하여 이 차원을 구현합니다.

재순위 지정자 (Déjean et al., 2024)³ 목록의 맨 위에

각 사용자 검색어에 대해 가장 유사한 문서 20개를 보여줍니다.

에이전트 기반 구현 에이전트 기반 RAG 시스템 은 필요에 따라 보다 적합한 맥락을 고려하려고 시도할 수 있습니다. 구체적으로, 에이전트는

추가 검색 라운드를 유발하고 적응할 수 있습니다.

시스템은 적절하다고 판단되는 방식으로 질의 형식을 지정하여, 보다 관련성 높은 맥락을 반복적으로 얻을 수 있도록 합니다.

우리는 마지막 재구성 버전을 기준으로 지표를 계산합니다.

에이전트가 RAG 도구에 사용하는 쿼리의 일부입니다.

이는 답변 생성 직전에 발생합니다.

실험 설정: 모든 쿼리를 실행합니다.

FIQA 및 CDQStack-English 테스트 세트에서

두 시스템에 대한 성능을 평가하기 위해

각각 QA 및 IR/E 작업을 수행했습니다. 우리는 그러지 않았습니다.

크기를 고려하여 NQ와 FEVER를 고려하십시오.

섹션 6.1 에 자세히 설명된 Agentic RAG는 다음과 같은 조치를 취할 것입니다.

각각 7일 이상 소요되었습니다.

평가 지표 재순위화를 평가하기 위해 NDCG@10을 사용하여 실제 링크와 비교합니다.

선택된 쿼리와 문서 간의 관계

하나. 측정 기준은 위에서 설명한 것과 동일합니다.

쿼리 재작성.

5.4 기본 LLM

정의: 에이전트 설정과 항상된 설정 모두

기본 LLM 모델 선택에 따라 결과가 크게 달라지는데, 모델마다 산출하는 결과가 다르기 때문입니다.

동일한 질문을 받았을 때에도 답변이 제공됩니다.

그리고 검색된 맥락. 더 나아가, 역할은 다음과 같습니다.

생성기는 Agentic RAG에서 특히 중요합니다.

여기서 모델은 최종 결과물을 만들어낼 뿐만 아니라

답변뿐만 아니라 각 단계에서 의사 결정도 내려야 합니다.

워크플로우. 우리는 이해하고자 합니다.

발전기 성능 저하가 미치는 영향을 정량화하는 것

시스템에서, 더 강력한 것과 비교했을 때 어떤 차이가 있는지를 보여줍니다. 그랬을 것이다.

항상된 에이전트 기반 구현을 통해 이러한 효과를 평가하기 위해

다양한 기능을 가진 네 가지 생성기를 테스트했습니다.

능력, 즉 Qwen3-0.6B (생각 없이)

Qwen3-4B, Qwen3-8B 및 Qwen3-32B (Yang et al., 2025). 우리는 발전기 용량을 다음을 기준으로 정의합니다.

표 6에 제시된 벤치마크를 기준으로 삼지 않습니다.

어떤 모델이 발전기로서 가장 우수한 성능을 보이는지 평가하는 것이 아니라, 전반적인 RAG 성능이 어떠한지 평가하는 것입니다.

서로 다른 출력의 발전기에 의해 영향을 받습니다. 우리는 쿼리 재작성 및 재순위 지정을 포함하는 항상된 RAG 설정과 에이전트 설정을 활용했습니다.

시스템 프롬프트는 부록 A.1의 그림 3 에 정의되어 있습니다.

실험 설정 모든 쿼리는 다음 환경에서 실행됩니다.

FIQA 및 CDQStack-English의 테스트 세트에 대한

QA 측면에서 성능을 평가하기 위한 두 가지 시스템

그리고 IR/E 과제를 각각 고려했습니다. 우리는 이를 고려하지 않았습니다.

크기 때문에 NQ와 FEVER가 발생합니다.

평가 지표 우리는 품질을 평가합니다

두 시스템에서 생성된 최종 답변은 다음과 같습니다.

LLM을 심사자로 활용하는 자동 측정 방식

패러다임. 제안된 여러 접근 방식 중에서

문헌 (Kim et al., 2024; Wang et al., 2025)

Es et al., 2024) Seleni-70B (Alexandru) 를 선택합니다.

³ <https://huggingface.co/naver/trecdl22-크로스인코더-일렉트라>

모델 제너럴	맞추다	이유 평균	
Qwen3 GPQA-D 이페발 에이미 '24			
0.6B	22.9	54.5	3.4 26.9
4B	55.9	81.9	73.0 70.3
8B	62.0	85.0	76.0 74.3
32B	68.4	85	81.4 78.3

표 6: 분석 대상으로 선정된 모델들의 성능

양 이 보고한 기본 LLM의 영향

et al. (2025). 선택된 벤치마크는 Graduate-Level Google-Proof QA Diamond (Rein et al., 2023)입니다.

평가 후 지침 (Zhou et al., 2023) 및 미국 초청 수학 시험

(AIME, 2024).

et al., 2025), Llama-3.3- 기반의 미세 조정 모델

70B-Instruct는 더 작은 버전이 높은 점수를 받는 것처럼 보입니다.

LLM 판사 과정의 최고 전문가들4

. 그만큼

저자들은 또한 이러한 모델의 판단이

재무 QA에 대한 인간 평가와 매우 유사합니다.

FIQA를 사용하는 점을 고려할 때 이는 중요한 과제입니다.

이 데이터셋의 경우, 우리는 가장 인간 친화적인 이항 0-1 분류 방식을 채택합니다.

정렬된 옵션을 선택하고 평균 점수를 보고합니다.

반면, CQADupStack-English의 경우,

그러한 정렬은 입증되지 않았습니다. 따라서 우리는 테스트 데이터의 하위 집합 (5%, 312개 답변)에 대해 수동 주석을 수행하여 이를 평가했습니다.

총 쌍의 수를 대상으로 일치율을 계산했습니다.

두 명의 인간 주석자와 자동 주석자 사이의

측정 기준. 우리는 쌍별 측정 기준(두 개가 주어졌을 때)을 선택합니다.

두 가지 다른 모델의 답변 중에서 가장 적합한 것을 선택하세요.

1). 주석자 간 일치율(시간 비율)

두 사람 모두 A 또는 B를 선택한 경우의 일치율은 71.9%이고,

사람과 모델 간의 일치율은 65.4%입니다. 수동 주석

한 쌍당 평균 1.5 분이 소요되었으며, 그 결과

총 15.5 시간이 소요되었습니다. 주석

지침은 부록에 제시되어 있습니다. 기본 모델 변경의 영향을 요약하면 다음과 같습니다.

더 큰 모델이 더 자주 나타나는 비율을 계산합니다.

더 작은 상대를 상대로 승리합니다.

6개 결과

사용자 의도 처리 결과는 표 3에 나와 있습니다.

사용자 의도 처리와 관련하여, 에이전트 설정이 향상된 설정보다 약간 더 우수한 성능을 보이는 것을 확인했습니다.

FIQA 및 CDQADupStack-EN 작업. 이 경우

발열의 경우, 후자가 전자보다 더 나은 성능을 보입니다.

마진 (F1 점수 +28.8 점)은 매우 낮은 재현율로 인해 발생했습니다.

(49.3). 이처럼 낮은 회상률은 다음과 같은 사실 때문입니다.

시스템은 검색을 사용하지 않아야 하는 경우에도 검색을 사용하는 경우가 많습니다.

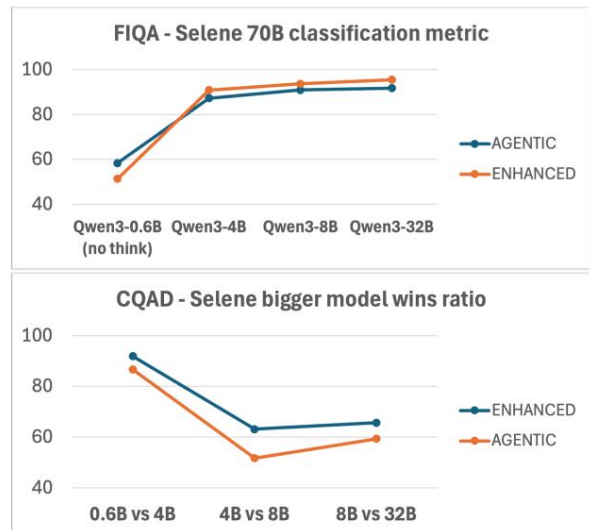


그림 2: 기본 LLM을 변경할 때 향상된 설정과 에이전트 설정의 성능. FIQA의 경우,

해당 지표는 분류 지표의 전체 비율입니다.

(답이 지시사항을 따르면 1, 그렇지 않으면 0)

CQADupStack-EN의 경우, 해당 지표는 쌍별 분석을 기반으로 하며, 다음 두 경우가 동시에 발생한 비율로 계산됩니다.

더 큰 모델의 답변이 더 작은 모델의 답변보다 더 낮다 상대측. 두 지표 모두 LLM-as-a-Judge(Selene-70B)를 통해 계산됩니다.

아닙니다. 이는 첫 번째와 두 번째 데이터 세트가 매우 명확한 도메인 정의(금융, 영어)를 가지고 있기 때문이라고 생각합니다.

문법적인 측면을 중점적으로 다루는 반면, FEVER 작업은 사용자 쿼리 검증을 목표로 하므로 설계상 훨씬 덜 엄격합니다.

사실 정보에 관한 것이므로 더 어려워집니다.

에이전트가 어떤 종류의 요청에 대한 검색이 필요한지 정확히 이해하도록 합니다.

표 4의 쿼리 재작성 결과는 다음과 같습니다.

에이전트 설정이 향상된 설정보다 성능이 더 좋습니다.

첫째, 평균 2.8 NDCG@10 포인트의 이득을 보였습니다.

우리는 이것이 전자의 유연성 덕분이라고 생각합니다.

수행 여부를 동적으로 결정할 수 있습니다.

쿼리 재작성 방법과 그 과정에 대해 설명합니다. 이는 쿼리 재작성이 항상 유익하며, 적응형 쿼리를 사용하는 것이 중요하다는 것을 시사합니다.

사안별로 어떻게 할지 결정하는 접근 방식

기초적인 접근 방식이 가장 신뢰할 만합니다.

향상된 설정에서 문서 목록 세분화는 재순위 지정을 통해 상당한 긍정적 영향을 미칩니다.

성능. 이와 대조적으로 Agentic RAG 설정은 성능이 더 뛰어납니다.

검색 단계를 반복해도 아무런 이점을 얻지 못합니다.

더 나은 문서를 식별할 수 없는 것 같습니다.

처음에 검색된 것들.

LLM의 기초 본 연구에서는 향상된 시스템과 에이전트 시스템이 모델 규모에 따라 뚜렷한 성능 패턴을 보이는지 평가하고자 합니다.

그림 2는 이를 다시 보여줍니다.

⁴<https://huggingface.co/blog/arena-atla>

두 데이터 세트에 대한 평균 점수를 출력합니다.

이는 두 시스템이 나타나지 않는다는 것을 보여줍니다.

변화 시 패턴에 상당한 차이가 발생합니다.

기본 LLM입니다. 실제로 FIQA의 성능 향상은 동일한 분포를 따릅니다.

향상된 RAG와 에이전트형 RAG도 마찬가지다.

더 큰 모델이 나타나는 시간의 비율에 대해서

CQADupStack-En 설정에서 더 작은 값보다 선호됩니다. 평가 지표에 대한 전체 결과는 다음과 같습니다.

자세한 내용은 부록 A.3에 보고되어 있습니다.

6.1 비용 및 시간

향상된 RAG 시스템과 에이전트 기반 RAG 시스템을 비교할 때 중요한 요소 중 하나는 계산 비용과 금전적 비용입니다. 실제 많은 환경에서 실무자들은 이러한 비용 차이를 합리적으로 받아들일 수 있습니다.

자연 시간이나 리소스 사용량을 줄일 수 있다면 성능이 약간 저하되더라도 괜찮습니다.

향상된 설정과 에이전트 설정에 대한 고정 시간 및 비용. 두 시스템은 일부 고정된 사항을 공유합니다.

하드웨어 요구 사항으로 인한 비용. 본 연구에서는, 두 환경 모두에서 AWS t3.large EC2 인스턴스를 활용했습니다.

벡터 검색 기능을 갖춘 관계형 데이터베이스를 인스턴스화 하려면 인스턴스 생성(주문형 사용 시 시간당 0.09달러)이 필요합니다.

pabilities (pgvector5). 우리는 RAG를 구현합니다.

t2.medium 환경에서 애플리케이션 백엔드 개발 (시간당 0.05달러).

우리는 자체 개발한 8x a40 클러스터 (46GB) 에서 오픈 LLM을 테스트했습니다. Qwen3 0.6B, 4B, 8B 버전을 실행했습니다.

단일 GPU 버전과 달리 32B 버전은 4개의 A40 칩을 사용합니다.

AWS에서 이와 유사한 설정은 g4ad.8xlarge일 수 있습니다.

시간당 1.9달러의 비용이 발생합니다. 검색과 관련된 시간과 비용은 두 설정 모두 동일합니다. 기본 임베더 모델로는 OpenAI text-embedding-3-small을 코사인 유사도와 함께 사용했습니다. 향상된 RAG

3억 개의 매개변수를 가진 모델을 사용하여 재순위 지정을 수행합니다.

LLM과 동일한 클러스터이므로 추가 비용은 다음과 같습니다.

무시할 만하다.

실행 시간 비용과 토큰 수. 우리는 실행 시간 RAG 비용을 토큰 수로 근사합니다.

처리된 입력 및 출력 토큰 수. 이 측정항목은

하드웨어에 구매받지 않으므로 비용을 추정할 수 있습니다.

모델의 처리량을 고려했을 때 모든 배포에 대해

그리고 시간당 요금제를 사용합니다. 토큰 사용량을 분석합니다.

향상된 환경과 에이전트 환경 간의 비용 차이를 정량화하기 위해

GPT 및 Qwen 모델을 모두 사용했습니다.

또한 중간 간 지연 시간(시작부터 종료까지 걸리는 시간)도 보고합니다.

질의를 접수하여 최종 답변을 반환하는 과정입니다.

LLM을 열면 지연 시간은 여러 요인에 의해 크게 영향을 받습니다.

기본 하드웨어. 표 9 (부록)는 다음을 보고합니다.

접근 방식별 소요 시간 및 토큰 요약.

FIQA의 경우, 에이전트 설정은 평균적으로 2.7배 더 많은 입력 토큰을 필요로 한다는 것을 알 수 있습니다.

Enhanced RAG보다 1.7배 더 많은 출력 토큰을 생성합니다.

CQADupStack-En의 경우 증가폭은 훨씬 더 커서 각각 3.9배와 2.0배입니다. 두 시나리오 모두에서,

소요 시간은 평균 1.5배 증가합니다. 이러한 차이는 곧 재정적 비용 증가로 이어집니다.

Enhanced RAG에서 유효한 쿼리의 경우 다음과 같은 결과를 얻습니다.

전체 시간의 약 45~50%는 답을 도출하는 데 소요되고, 비슷한 비율의 시간이 다른 작업에 소요됩니다.

쿼리 재작성으로 인해 검색 효율이 0~5% 감소하고, 결과 처리 효율은 0~2% 감소합니다.

문서 재순회. 자연 시간의 주요 요인은 LLM 호출입니다. 따라서 모든 성능은

최적화는 주로 그 점에 초점을 맞춰야 합니다.

7. 결론

우리의 실험적 비교 결과는 둘 다 아님을 보여줍니다.

향상된 RAG와 에이전트형 RAG 중 어느 것도 보편적으로 우월하지 않습니다.

첫째, 사용자 행동이 고도로 구조화된 좁고 명확하게 정의된 영역에서 에이전트 RAG는 사용자 의도를 처리하는 데 탁월하다는 것을 알 수 있습니다.

사용자 쿼리를 이해하는 능력에 달려 있습니다. 하지만 더 넓거나 잡음이 많은 영역에서는 당사의 향상된 기능이 더 효과적입니다.

RAG 라우팅 시스템이 더 신뢰할 수 있음이 입증되었습니다. 둘째, 쿼리 정렬과 관련하여 문서의 구조와 의미론

KB, Agentic RAG는 Enhanced RAG보다 우수한 성능을 보입니다.

검색 품질. 쿼리 재작성의 동적 활용.

더욱 관련성 높은 맥락을 검색할 수 있도록 해줍니다. 셋째,

Agentic RAG가 특정 항목을 선택할 때 다음과 같은 사실을 발견했습니다.

문서의 경우, 재순위 지정 만큼 좋지는 않습니다.

Enhanced RAG를 통해 가장 의미 있는 문서만 선별했습니다. 저희 결과는 통합이 중요하다는 것을 시사합니다.

Agentic 파이프라인에 명시적인 재순위 지정 단계

상당한 이득을 가져다줄 수 있습니다. 넷째, 우리는 다음과 같이 관찰합니다.

기본 LLM을 변경하면 다음과 같은 결과가 나타납니다.

두 설정 모두에서 성능 변화가 동일합니다.

모델 크기의 영향은 다음과 같은 유사한 경향을 보입니다.

두 가지 패러다임 모두: 기본 LLM이 됨에 따라 크기가 커질수록 성능 향상률은 비슷합니다.

비용 분석 결과 Agentic RAG는 다음과 같은 특징을 보입니다.

체계적으로 더 비쌌 - 최대 3.6배

추가적인 추론 단계와 반복적인 도구 호출로 인해 비용이 더 많이 듭니다. 이러한 비용 차이는 발생해서는 안 됩니다.

간과될 수 있는 것: 잘 최적화된 향상된 RAG

Agentic의 성능과 동등하거나 그 이상을 달성할 수 있습니다.

더욱 효율적인 상태를 유지합니다.

⁵<https://github.com/pgvector/pgvector>

제한 사항

본 연구에는 몇 가지 한계점이 있으며, 이러한 한계점을 고려해야 합니다. 결과 해석 시 고려되는 사항은 다음과 같습니다. 첫째, 저희는 구현 버전을 공개하지 않습니다. PocketFlow 기반 에이전트는 완전한 재현성을 제한할 수 있습니다 . 둘째, 저희 평가에서는 RAG 행동의 주요 차원과 문서 요약과 같은 추가적인 측면을 다룹니다.

문서 재포장(문서 재분류)

중요도에 따른 맥락적 고려 및 출력 정제는 고려되지 않았습니다. 또한, 저희 에이전트는 단 하나의 도구만 갖추고 있지만, 더 풍부한 에이전트 설정은 다양한 동작을 보일 수 있습니다. 전반적으로, 각 블록은

향상된 RAG는 이전 연구를 기반으로 구현되었지만 , 다른 접근 방식과의 비교가 부족하여 기존 방식보다 더 나은 성능을 보이는 다른 접근 방식이 존재하지 않습니다. 우리가 선택했습니다.

참고 자료

AIME. 2024. Aime의 문제점과 해결책. https://artofproblemsolving.com/wiki/index.php/AIME_문제_및_해결_방법. 2024년 접속.

안드레이 알렉산드루, 안토니아 칼비, 헨리 브룸필드, 잭슨 골든, 카일 다이, 마티아스 레이스, 모리스 Burger, Max Bartolo, Roman Engeler, Sashank Pisupati , Toby Drane, 박영선. 2025. [아틀라 셀레네 미니: 범용 평가 모델](#). 사전 출판본, arXiv:2501.17195.

아마존 웹 서비스. 2025. 아마존 웹 서비스 — 기반암. <https://aws.amazon.com/bedrock/>

무하마드 아르슬란, 후삼 가넴, 사바 무나와르, 그리고 크리스토프 크루즈. 2024. [rag에 대한 설문조사 일러스](#). 프로세디아 컴퓨터 과학, 246:3781–3790. 제28회 지식 기반 국제 컨퍼런스 지능형 정보 및 엔지니어링 시스템 (2024년 한국 경제개발원(KES) 기준).

BrainBlend-AI. 2025. Brainblend-ai/원자 에이전트: 에이전트 구축을 위한 모듈형 에이전트 프레임워크 원자적 구성 요소에서 가져온 것입니다. <https://github.com/브레인블렌드-AI/아토믹-에이전트>. MIT 라이선스; 접속일 : 2025년 11월 18일.

앤드류 첸, 앤디 초우, 애런 데이비스, 아르준 DCunha, Ali Ghodsi, Sue Ann Hong, Andy Konwinski , Clemens Mewald, Siddharth Murching, Tomas Nykodym, 폴 오길비, 마니 파크헤, 아베쉬 싱, Fen Xie, Matei Zaharia, Richard Zang, Juntao Zheng, 및 Corey Zumar. 2020. [mlflow의 발전: A 마신러닝 수명주기를 가속화하는 시스템](#). 제4차 국제 워크숍 회의록 엔드투엔드 마신러닝 을 위한 데이터 관리 관련 DEEM '20 학회, 뉴욕, 미국. 컴퓨팅 머신용.

CrewAllInc 기여자. 2025a. crewAllInc/crewAI: 자율적인 역할극 AI 에이전트를 조율하기 위한 프레임워크 . <https://github.com/crewAllInc/crewAI>. 접속일: 2025년 11월 18일.

Pydantic 기여자. 2025b. pydantic/pydantic-ai: pydantic 방식의 Genai 에이전트 프레임워크. <https://github.com/pydantic/pydantic-ai>. 접속일: 2025년 11월 18일.

구스타보 데 아키노 에 아키노, 나딜라 다 실바 데 아제베도, 레안드로 유이티 실바 오키모토, 레오나르도 Yuto Suzuki Camelo, Hendrio Luis de Souza Bragança, Rubens Fernandes, Andre Printes, Fábio Cardoso , Raimundo Gomes 및 Israel Gondres Torné. 2025년. [누더기 시스템에서 다중 에이전트 시스템으로: 개요 LLM 개발에 대한 현대적 접근 방식](#). 논문 초고.

Hervé Déjean, Stéphane Clinchant, and Thibault For-mal. 2024. [Splade의 재순위를 위한 교차 인코더와 llms의 철저한 비교](#). 출판 전 논문, arXiv:2403.10407.

샤홀 에스(Shahul Es), 지틴 제임스(Jithin James), 루이스 에스피노사 안케(Luis Espinosa Anke), 스티븐 쇼카에르트. 2024. [RAGAs: 검색 증강 생성의 자동 평가](#). 제18회 유럽 전산언어학회 학술대회 논문집 : 시스템 시연, 150~158페이지, 세인트 줄리안스, 몰타. 전산언어학 협회.

Wenqi Fan, Yajuan Ding, Liangbo Ning, Shijie Wang, Hengyun Li, Dawei Yin, Tat-Seng Chua 및 Qing Li. 2024. [rag 회의 llms에 대한 설문 조사: Towards 검색 기능이 강화된 대규모 언어 모델](#). 제30회 ACM SIGKDD 컨퍼런스 회의록 에서 지식 발견 및 데이터 마이닝, KDD '24 페이지 6491–6501, 뉴욕, 뉴욕, 미국. 협회 컴퓨팅 머신용.

Luyu Gao, Xueguang Ma, Jimmy Lin 및 Jamie Callan. 2023a. [관련성 레이블 없이 정확한 제로샷 밀집 검색](#) . 제61회 연례 학술대회 논문집 전산언어학회 회의록 (제1권: 장편 논문), 1762–1777쪽 토론토, 캐나다. 전산언어학회 .

Yunfan Gao, Yun Xiong, Xinyu Gao, Kangxiang Jia, Jin-liu Pan, Yuxi Bi, Yi Dai, Jiawei Sun, Meng Wang 및 Haofen Wang. 2023b. 대규모 언어 모델을 위한 [검색 증강 생성 : 설문 조사](#). 출판 전 논문, arXiv:2312.10997.

도리스 후게빈(Doris Hoogeveen), 카린 M. 베르스포르(Karin M. Verspoor), 티모시 볼드윈. 2015. [Cqadupstack: 벤치마크 데이터 커뮤니티 질문 답변 연구를 위해 설정되었습니다](#). ~ 안에 제20회 호주-뉴질랜드 문서 컴퓨팅 심포지엄(ADCS '15) 회의록, 뉴욕, 뉴욕, 미국. 컴퓨터 학회(Association for Computing Machinery).

황이정(Yizheng Huang)과 지미 황(Jimmy Huang). 2024. [설문조사 대규모 언어 모델을 위한 검색 강화 텍스트 생성에 관하여](#) . 사전출판본, arXiv:2404.10981.

IBM. 2025. IBM WatsonX — RAG 개발. <https://www.ibm.com/it-it/products/watsonx-ai/rag-개발>.

Kalervo Järvelin과 Jaana Kekäläinen. 2002. IR 기술의 누적 이득 기반 평가. ACM Trans. Inf. Syst., 20(4):422–446.

김승원, 신자민, 조예진, 장조엘, Shayne Longpre, 이화란, 윤상두, 신성진, 김성동, 제임스 손, 서민준. 2024. **프로메테우스: 언어 모델에 세밀한 평가 가능 유도**.

사전출판본, arXiv:2310.08491.

톰 크비아트코프스키, 제니마리아 팔로마키, 올리비아 레드 필드, 마이클 콜린스, 안쿠르 파리크, 크리스 알베르티, Danielle Epstein, Illia Polosukhin, Jacob Devlin, Kenton Lee, Kristina Toutanova, Llion Jones, Matthew Kelly, 멩웨이 창, 앤드류 M. 다이, 야콥 Uszkoreit, Quoc Le, and Slav Petrov. 2019. **자연어 질문: 질문 답변의 기준점** 연구. 전산언어 학회 논문집, 7:452–466.

LangChain Inc. 2025. langchain-ai/langgraph: 그래프를 기반으로 탄력적인 언어 에이전트 구축. <https://github.com/langchain-ai/langgraph>. MIT 라이선스; 접속일: 2025년 11월 18일.

패트릭 루이스, 에단 페레즈, 알렉산드라 피크투스, 파비오 Petroni, Vladimir Karpukhin, Naman Goyal, Hein-rich Küttler, Mike Lewis, Wen-tau Yih, Tim Rock-täschel, Sebastian Riedel 및 Douwe Kiela. 2020. **지식 집약적인 자연어 처리 작업을 위한 검색 증강 생성**. 신경 정보 처리 시스템의 발전(Advances in Neural Information Processing Systems), 33권, 9459쪽~9474. 커런 어소시에이츠 주식회사

제리 류. 2022. **라마인덱스**.

마 신베이, 공 예윤, 허 핑청, 자오 하이, 및 Nan Duan. 2023. **검색 증강 대규모 언어 모델에서의 쿼리 재작성**. 다음 논문집에서 2023 자연어 처리의 경험적 방법론에 관한 학술대회, 5303–5315쪽, 싱가포르. 전산언어학회.

아만 마단, 니켓 탠던, 프라카르 굽타, 스카일러 Hallinan, Luyu Gao, Sarah Wiegrefe, Uri Alon, 누하 디지리, 슈리마이 프라부모예, 양이밍, 샤생크 굽타(Shashank Gupta), 보살 프라사드 마쭈더(Bodhisattwa Prasad Majumder), 캐서린 헤르만, 셴 웰렉, 아미르 야즈단 바크시, 피터 클라크. 2023. 자기 피드백을 통한 정제. **학술대회 논문집 제37회 국제 신경 정보 학회 프로세싱 시스템, NIPS '23**, 미국 뉴욕주 레드록. 커런 어소시에이츠 주식회사

마케도 마리아, 지그프리트 한트슈, 안드레 프레이타스, 브라이언 데이비스, 로스 맥더모트, 마넬 자루크, 그리고 알렉산드라 발라후르. 2018. **WWW'18 오픈 챌린지: 금융 시장 의견 분석 및 질의응답**. ~ 안에 웹 컨퍼런스 부록 논문집 2018, WWW '18, 1941-1942페이지, 공화국 및 제네바 주, CHE. 국제 세계 웹 컨퍼런스 운영 위원회.

톨라 마스터맨, 산디 베센, 메이슨 소텔, 알렉스 차오. 2024. **신형 AI 에이전트의 현황** **추론, 계획 및 도구 호출을 위한 아키텍처: 설문조사**. 사전출판본, arXiv:2404.11584.

마이크로소프트. 2025. microsoft/autogen: Autogen – 에이전트 오케스트레이션 및 도구 호출. <https://github.com/microsoft/autogen>. MIT 라이선스; 접속일: 2025년 11월 18일.

Microsoft Azure. 2025. Azure AI 검색 - rag 솔루션 튜토리얼. <https://docs.azure.cn/en-us/검색/튜토리얼-rag-build-solution>.

프누 네하(Fnu Neha)와 딥시카 바티(Deepshikha Bhati). 2025. **전통 rag vs. agentic rag: 검색 증강 시스템에 대한 비교 연구**. Authorea Preprints.

Zhen Qin, Rolf Jagerman, Kai Hui, Honglei Zhuang, 우 준루, 르 안, 심 자밍, 리우 티안치, 지아루 Liu, Donald Metzler, Xuanhui Wang, Michael Bendersky. 2024. **대규모 언어 모델은 쌍별 순위 지정 프롬프트를 통해 효과적인 텍스트 순위 지정 도구입니다**. ~ 안에 전산언어학회(NAAACL) 2024 연구 결과 보고서, 1504~1518쪽, 멕시코

멕시코시티. 전산언어학 협회 .

데이비드 레인, 베티 리 후, 아사 쿠퍼 스틱랜드, 잭슨 페티, 리처드 위안저 팡, 줄리앙 디 라니, 줄리안 마이클, 사무엘 R. 보우먼. 2023. **Gpqa: 대학원 수준의 구글 검색에도 문제없는 질의응답 기준점**. 사전출판본, arXiv:2311.12022.

아이메릭 루세, 알베르트 빌라노바 델 모랄, 토마스 볼프, 레안드로 폰 베라, 에릭 카우니스마키. 2025. 'smolagents': smol 라이브러리 구축 훌륭한 에이전트 시스템. <https://github.com/허깅페이스/스몰에이전트>.

데벤드라 사찬, 마이크 루이스, 만다르 조시, 아르멘 Aghajanyan, Wen-tau Yih, Joelle Pineau 및 Luke Zettlmoeyer. 2022. **문단 검색 개선** **제로샷 질문 생성**. 다음 논문집에서 2022년 자연어 처리 분야의 경험적 방법론에 관한 학술대회, 3781~3797쪽, Abu 아랍에미리트 아부다비. 전산언어학 협회 .

노아 신, 페데리코 카사노, 애쉬윈 고포나스, Karthik Narasimhan, Shunyu Yao. 2023. **Re-flexion: 언어적 강화를 갖춘 언어 에이전트 학습**. 제37회 국제 학술대회 논문집 신경 정보 처리 시스템 학회 (NIPS '23), 미국 뉴욕주 레드록. Curran Associates Inc.

Aditi Singh, Abul Ehtesham, Saket Kumar 및 Tala Talaie Khoei. 2025. **에이전트 검색 - 증강** **세대: 행위 주체적 래그에 대한 조사**. 출판 전 논문, arXiv:2501.09136.

Weiwei Sun, Lingyong Yan, Xinyu Ma, Shuaiqiang Wang, Pengjie Ren, Zhumin Chen, Dawei Yin 및 런자오춘. 2023. **ChatGPT는 검색에 능한가?** **대규모 언어 모델을 재순위로 조사**

자차령 대표. 2023년 컨퍼런스 회의록에서 자연어 처리의 경험적 방법론, 14918~14937쪽, 싱가포르. 협회

계산언어학.

난단 타쿠르, 닐스 라이머스, 안드레아스 뢰클레, 아브히셰크 스리바스타바, 이라나 구레비치. 2021. [베이트: 제로샷 평가를 위한 이중 벤치마크 정보 검색 모델](#). 다음 논문집에서 신경 정보 처리 시스템 트랙에서 데이터셋 및 벤치마크, 1권.

제임스 손, 안드레아스 블라코스, 크리스토스 크리스토폴로폴로스, 아르피트 미탈. 2018. [FEVER: 사실 추출을 위한 대규모 데이터 세트 및 검증](#). 2018년 학술대회 논문집 북미 지부 회의 전산언어학회: 인간 언어 기술, 제1권 (장문) (논문), 809~819쪽, 뉴올리언스, 루이지애나. 전산언어학 협회.

PeiFeng Wang, Austin Xu, Yilun Zhou, Caiming Xiong, 그리고 샤피크 조티. 2025. [직접 판단 선호도 최적화](#). 2025년 컨퍼런스 회의록 자연어 처리의 경험적 방법론에 관한 학술 대회 논문집, 1979~2009년, 중국 쑤저우, 협회. 전산언어학 분야를 위해서.

Xiaohua Wang, Zhenghua Wang, Xuan Gao, 페이난 Zhang, Yixin Wu, Zhibo Xu, Tianyuan Shi, Zhengyuan Wang, Shizheng Li, Qi Qian, Ruicheng Yin, Changze Lv, Xiaoqing Zheng, Xuanjing 황. 2024. [모범 사례 탐색 검색 증강 생성](#). 다음 논문집에서 2024년 자연어 처리 분야 경험적 방법론 학술대회, 17716~17736 쪽, 마이애미, 플로리다, 미국. 전산 언어 처리 협회

언어학.

An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, Chengen Huang, Chenxu Lv, Chujie Zheng, Day-iheng Liu, Fan Zhou, Fei Huang, Feng Hu, Hao Ge, Haoran Wei, Huan Lin, Jialong Tang 및 41 기타. 2025. [Qwen3 기술 보고서](#). 출판 전 논문, arXiv:2505.09388.

Jeffrey Zhou, Tianjian Lu, Swaroop Mishra, 싯다르타 브라마(Brahma), 수조이 바수(Sujoy Basu), 이루안(Yi Luan), 데니 저우(Denny Zhou), Le Hou. 2023. [지시사항 이행 평가 대규모 언어 모델](#). 사전출판본, arXiv:2311.07911.

부록 예시

A.1 에이전트 RAG에 대한 프롬프트

Agentic RAG는 세 개의 노드로 정의됩니다. 오케스트레이터, 응답 노드 및 RAG 노드. 여기서는 오케 스트레이터(그림 3 및 4)와 응답 노드(그림 5) 에서 사용되는 프롬프트를 보고하는 반면, RAG 노드는 특정 프롬프트를 가지고 있지 않습니다. 시스템 메시지.

	CQAD	피카	
Qwen은 에이전트를 강화했습니다.			
0.6B 51,6 15,4 4B 95,7 81,8		51,4	58,4
8B 94,9 88,8 32B 94,5 91,3		90,9	87,3
		93,7	91
		95,5	91,8

표 7: 셀레네-70B를 기반으로 한 분류 기준 FIQA.

A.2 라우팅 시스템 세부 정보

우리가 구현하는 라우팅 시스템의 구성도 항상된 RAG 설정에 대한 설명은 8절에 나와 있습니다. 라우팅 메커니즘은 예제 기반 방식에 의존합니다. 분류. 쿼리는 유효한 참조 집합과 유효하지 않은 참조 집합, 두 가지 참조 집합과 비교됩니다. 쿼리가 유효한 집합과 충분히 유사하면 처리됩니다. 그렇지 않으면 거부됩니다. 핵심 과제는 "충분히 유사하다"는 것이 무엇을 의미하는지, 즉, 적절한 유사도 임계값을 선택합니다.

임계값 선택 임계값 선택은 다음과 같습니다. 다양한 방식으로 접근함 (예: 무작위 검색, 선형 모델, 분류 알고리즘). 클래스가 정의는 명확하고 잘 구분되어 있으며, 임계값이 있습니다. 쿼리가 자연스럽게 올바른 클래스 주변에 밀집 되므로 튜닝의 중요성이 줄어듭니다 . 하지만 실제로는 클래스 경계에 노이즈 가 포함되는 경우가 많습니다. 임계값을 필수적인 안전장치로 만드는 것 오분류.

A.3 LLM 평가의 기본 지표

다음 방법을 통해 계산된 지표에 대한 결과는 다음과 같습니다. Selene-70B는 표 7 (FIQA)에 보고되어 있습니다. 그림 7 (CQADupStack-EN).

프레임워크 장점 스물	에이전트 최	단점
소; 코드에이전트		저수준 추상화 부족; 제어 및 보안 수준 저하 코드에이전트로 인한 문제(처리 가능)
랭그래프	그래프 추상화; 다양한 통합 기능; 인기 있는 패턴 구현을 위한 다양한 자료	
라마인덱스	다양한 통합 기능과 인기 있는 패턴을 구현하기 위한 다양한 리소스가 제공됩니다.	높은 복잡성을 가진 코드베이스
포켓플로우 크루	최소한의 그래프 추상화	구현을 수행해야 합니다
AI		다중 에이전트에 집중
autogen	성숙한 생태계 유형의 안전	높은 복잡성
pydanticAI	성; 파이썬 기반	개발 초기 단계이므로 몇 가지 학습이 필요합니다. 개념
원자 에이전트 모듈형		개발 초기 단계이며, 관련 문서가 제한적입니다.

표 8: 에이전트 기반 RAG 구현을 위해 고려된 프레임워크 비교. "장점"과 "단점"은 다음과 같이 정의됩니다. 본 연구의 범위는 단일 RAG 도구를 사용하여 가능한 가장 간단한 에이전트를 구축하는 것입니다.

		피카				CQAD-EN							
모델		시간	토크 토큰		비율(Ag/En) 압력 출		시간	총 토크 비율(Ag/En)					
		입력 시간 입력 출력 1683 465				입력 출력 시간 입력 출력							
GPT-4.1-나노	En 9.0			1.1	2.2	0.8	7.0	856 331		1.2	3.0	0.9	
	Ag 10.2	3676 348	En 8.1				1743 236	8.6	2463 297				
퀘인3-0.6B				2.2	2.9	4.1	8.1	862 254		1.1	3.5	3.4	
	Ag 22.1	4978 979	En 35.5				1743	8.9	3032 867				
퀘인3-4B	1435	Ag 38.6	4834 1704	En 58.5	1.1	2.8	1.2	31.0	862 1019		1.2	3.9	1.9
	1743	1490						37.2	3372 1943				
퀘인3-8B				1.2	2.8	1.2	58.4	862 1339		0.9	3.9	1.5	
	Ag 69.9	4943 1837	En 62.6				1743	54.3	3394 1983				
퀘인3-32B	1695			1.5	2.7	1.1	43.9	862 1109		2.3	3.9	1.5	
	Ag 93.8	4766 1866						101.9	3359 1636				
평균 비율				1.5	2.7	1.7				1.4	3.6	1.8	

표 9: 비용(입력 및 출력 토큰 수로 측정) 및 시간(종단 간 지연 시간) 분석
사용자가 쿼리를 실행할 때 경험하는 내용입니다.) 제시된 비율은 에서 로 이동하는 데 필요한 곱셈 계수를 나타냅니다.
에이전트 설정으로 강화되었습니다. Qwen3-0.6B는 사고 모드 없이 실행되므로 실행 시간이 상당히 단축됩니다.
더 큰 Qwen 모델과 비교했을 때 출력 결과는 다음과 같습니다. Agentic RAG는 항상 최대 3 턴을 수행했습니다. 시나리오에서
더 많은 턴이 필요하게 되면 에이전트가 소모하는 토큰이 증가합니다.

```
## Context
You are a powerful agentic AI agent. Your task is to {task_description}

## Inputs
• query: {query}
• previous_tool_results: {previous_tool_results}

## Guidelines
To do complete your task, you can take several actions:
• answer: {answer_node_description}
• document_retrieval: {retrieval_node_description}

## Important
• Don't repeat actions unnecessarily.
• When composing the query for RAG search you must always write a passage to answer the input query provided by the user.
• If you think that the retrieved documents in the previous_tool_results can be further improved, you can rewrite the query and call document_retrieval again.
• If you see the relevant retrieved documents already in the previous_tool_results, you should proceed with answering!

## Metadata:
• current reasoning step: {current_reasoning_step}
• maximum reasoning steps: {max_reasoning_steps}
```

그림 3: 모든 작업에 대한 에이전트를 정의하는 시스템 프롬프트 템플릿.

FIQA:
task_description:
help users on financial issues using the tools at your disposal.

answer_node_description:
provide the final answer to the user, utilizing the retrieved documents if necessary. Choose this action when you have gathered enough information from the previous_tool_results.

retrieval_node_description:
retrieve documents from a knowledge base of financial information. Choose this to ground answers on external sources.

FEVER:
task_description:
help users verifying their queries on factual knowledge. For each user claim, you determine if it is supported or refuted by the world-knowledge on the topic.

answer_node_description:
provide the final answer to the user, utilizing the retrieved documents if necessary. Choose this action when you have gathered enough information from the previous_tool_results.

retrieval_node_description:
retrieve documents from a knowledge base of factual information. Choose this to ground answers on gold-standard knowledge. Be careful, your knowledge might be outdated.

NQ:
task_description:
help users using the tools at your disposal.

answer_node_description:
provide the final answer to the user, utilizing the retrieved documents if necessary. Choose this action when you have gathered enough information from the previous_tool_results.

retrieval_node_description:
retrieve documents from a knowledge base of potentially useful information. Choose this to ground answers on external sources.

COADUPSTACK-EN:
task_description:
help users finding blog posts about english grammar issues that are related to the user query. You have to find the relevant documents, summarize them, and inform the user about them.

answer_node_description:
provide a summary of the retrieved blog posts to the user. Choose this action when you have gathered enough information on grammar blog posts from the previous_tool_results.

retrieval_node_description :
retrieve blog posts from a knowledge base of English grammar issues. Choose this to find relevant blog posts. Never use it for queries not related to english grammar issues.

그림 4: 선택된 네 가지 작업 각각을 정의하는 시스템 프롬프트 입력 값.

FIQA:

Context

You are a blog posts summarizer. You'll be provided the original user query, a list of tools called and their result, a draft answer, and conversation history.

Provide a bullet point list of the relevant blog posts related to the user query. One bullet point per blog post.

Each bullet point is composed of 1) a short title (max 10 words); 2) a one sentence summary (max 15 words) of the blog post; 3) the reference to the document.

You do not answer the question. Instead, you provide an overview of the retrieved blog posts to the user.

Your objective is to make the user aware of the relevant blog posts related to their query.

Inputs

- question: {query}
- previous_tool_results: {previous_tool_results}
- draft answer: {draft_answer}

Guidelines

- Be consistent with previous responses in the conversation
- Introduce the bullet point list with a short sentence like "Here are some blog posts that might be relevant to your query:"
- Do not answer the question directly. Your goal is to summarize the retrieved blog posts.

FEVER:

Context

You are a question-answering assistant. You'll be provided the original user query, a list of tools called and their result, a draft answer, and conversation history. Provide a summary of the grounding reasons and evidence.

Inputs

- question: {query}
- previous_tool_results: {previous_tool_results}
- draft answer: {draft_answer}

Guidelines

- Be consistent with previous responses in the conversation.
- If a follow-up question references a previous conversation, use the conversation history to provide context.
- Summarize the reasons that support or contradict the statement.

FIQA & NQ:

Context

You are a question-answering assistant. You'll be provided the original user query, a list of tools called and their result, a draft answer, and conversation history.

Provide the final answer to the user.

Inputs

- question: {query}
- previous_tool_results: {previous_tool_results}
- draft answer: {draft_answer}

Guidelines

- Be consistent with previous responses in the conversation.
- If a follow-up question references a previous conversation, use the conversation history to provide context.

그림 5: 답변 생성 과정을 정의하는 답변 노드 시스템 프롬프트.

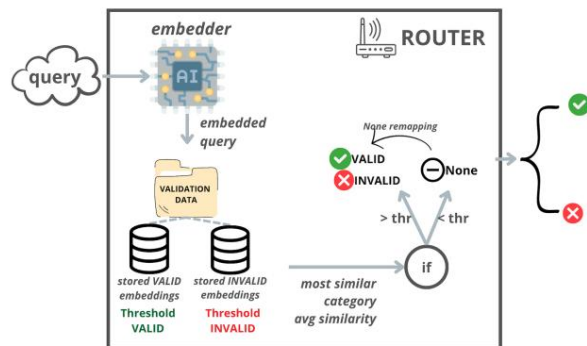


그림 6: 향상된 라우팅 시스템에 사용되는 스키마. 우리는 유효성 검사 세트에서 가져온 임베딩 예제를 각각 사용하여 유효한 쿼리와 유효하지 않은 쿼리, 이렇게 두 가지 쿼리 클래스를 정의합니다. 라우터는 들어오는 각 쿼리를 임베딩하고, 코사인 유사도를 통해 가장 유사한 상위 20개 예제를 검색한 후, 평균 유사도가 가장 높은 클래스를 선택합니다. 선택된 클래스의 평균 유사도가 미리 정의된 임계값을 초과하면 해당 레이블이 할당되고, 그렇지 않으면 라우터는 None을 반환합니다. None은 비즈니스 로직에 따라 추가적으로 재매핑될 수 있습니다.

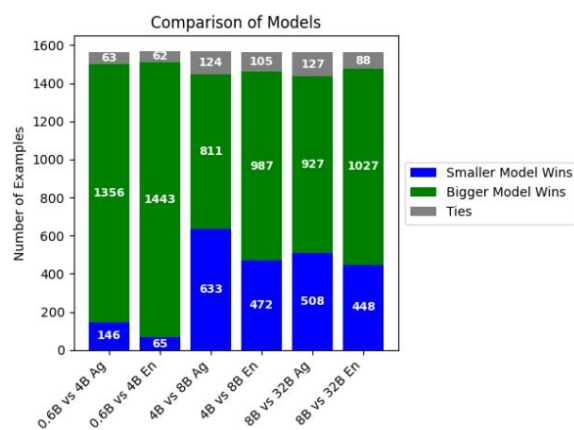


그림 7: CQADupStack-EN 에 대한 Selene-70B 기반 쌍별 측정 지표 .

GENERAL DESCRIPTION

You are evaluating a system that, given a user query, retrieves a list of similar queries previously submitted by other users. These past queries are called documents.

The system's output must be a list of bullet points, each representing one and only one document.

An ideal bullet point contains:

- a title of at most five words,
- a brief description of the retrieved document, and
- a reference to the document number.

Important: The documents do not contain answers to the user's query; they are actual queries submitted in the past. If the system's response includes a document description that explains the grammar question rather than restating the query itself, this is incorrect.

ANNOTATION

For each query, there are two system responses, A and B. Your task is to select the better response based on the criteria above.

If you are unsure, you may write "I don't know." If both responses are of equal quality, write "same."

Common situations include:

- The document description contains an answer rather than the original query (serious error).
- One of the listed documents is irrelevant to the query (serious error).
- If the two responses differ in the number of documents and all documents are relevant, always choose the one with more documents.
- The response mixes the document number with the title (error).
- The document description is excessively long (error).
- The document number reference is missing (error).

그림 8: CQADupStack-English 평가 지침. 이 지침은 인간 주석자와 LLM-as-a-Judge(Selene 모델) 모두에서 사용됩니다.

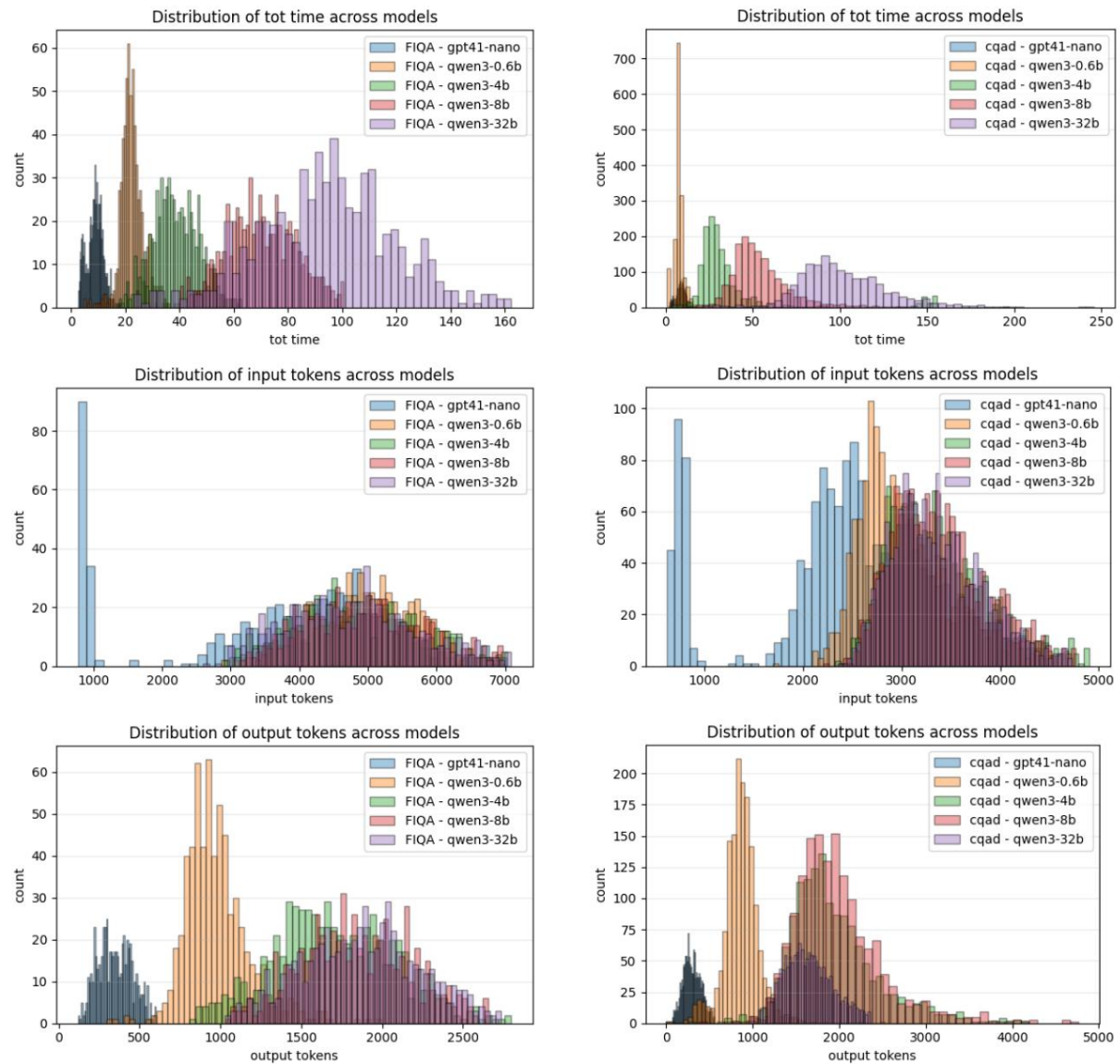


그림 9: 에이전트 환경에서 단일 사용자 쿼리를 처리할 때 각 모델의 전체 계산 비용 및 토큰 사용량. Qwen3 4B, 8B 및 32B는 사고 모드로 작동하는 반면, 0.6B 변형은 사고 모드 없이 실행됩니다.

Qwen3 0.6B, 4B, 8B는 단일 NVIDIA A40에서 실행되는 반면, Qwen3 32B는 4개의 A40 GPU에서 실행됩니다.

상단: 시스템이 쿼리를 수신한 순간부터 최종 답변이 생성되는 순간까지 측정된 총 지연 시간 분포. Qwen 제품군 중에서 Qwen3-0.6B는 크기가 작고 사고 모드가 없기 때문에 가장 낮은 지연 시간을 달성합니다. 중간: 평균 입력 토큰 수. 이 값은 모델 크기가 커질수록 약간 증가합니다. GPT-4.1-nano의 입력 토큰 수가 적은 피크는 모델이 RAG 도구를 자주 사용하지 않아 필요한 추론 단계 수를 줄이기 때문에 발생합니다.

하단: 평균 출력 토큰 수. 여기서 가장 큰 차이가 나타납니다. 사고 모드를 활성화하면 Qwen3 4B, 8B, 32B가 훨씬 더 긴 출력을 생성합니다.